Green Pace

Security Policy Presentation

Developer: *Jessica Johnson*

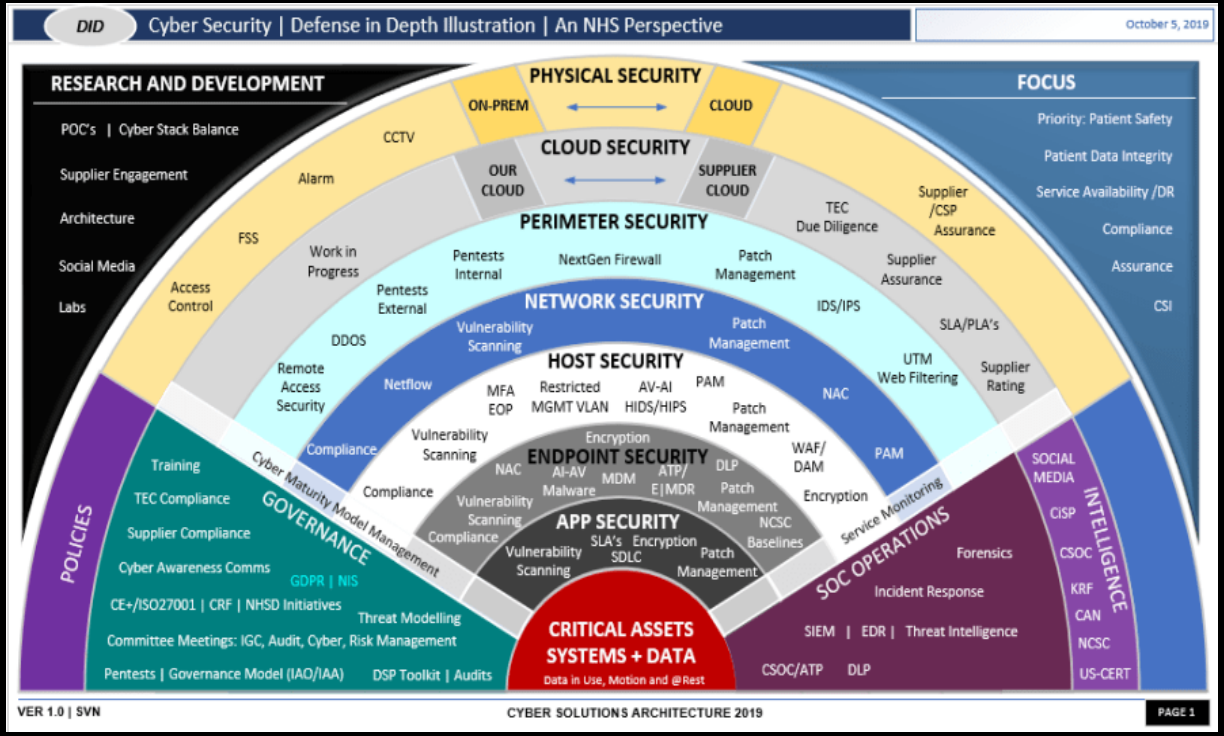
02/21/2026

Green Pace

OVERVIEW: DEFENSE IN DEPTH

Security Policy Overview

- Vulnerabilities identified through external testing
- Automated attack exposure
- Defense-in-depth implementation
- Secure coding policy enforcement



Green Pace

THREATS MATRIX

Likely	Priority
✓ SQL Injection ✓ Input Validation Failures ✓ Weak Authentication ✓ Cross-Site Scripting (XSS)	✓ Sensitive Data Exposure ✓ Missing Encryption (At Rest/ In Transit) ✓ Privilege Escalation ✓ Broken Access Control
Low priority	**Unlikely**
✓ Insufficient Logging ✓ Verbose Error Messaging ✓ Minor Configuration Weaknesses ✓ Session Timeout Issues	✓ Physical Server Access ✓ Advanced Persistent Threat ✓ Insider Collusion Attack ✓ Nation-State Level Attack

Green Pace

SECURE CODING PRINCIPLES

Principle	How its Applied
Input Validation	Parameterized queries, input sanitization
Least Privilege	Role Based access control
Defense in Depth	Layered authentication and encryption
Fail Securely	Default deny access
Secure Defaults	Enforce MFA by default
Encryption Enforcement	TLS and AES-256
Secure Session Management	Session Expiration and Token Validation
Logging and Monitoring	Centralized audit logs
Strong Authentication	Password Hashing
Error Handling	Generic user-facing messages

Green Pace

CODING STANDARDS

1. Validate and sanitize all external input
2. Use parameterized queries for database access
3. Enforce role-based access control
4. Implement multi-factor authentication
5. Encrypt data in transit (TLS 1.2+)
6. Encrypt data at rest (AES-256)
7. Enforce secure session expiration and token validation
8. Log and monitor authentication attempts
9. Hash passwords using bcrypt
10. Use generic error messages

Green Pace

ENCRYPTION STRATEGY

Data in Transit

- TLS 1.2 or higher
- HTTPS enforced
- Secure API communication

Data at Rest

- **AES-256 encryption**
- **Encryption databases**
- **Encrypted backups**

Data in Use

- **Role-based access control**
- **Memory protection practices**
- **Minimized data exposure**

Green Pace

TRIPLE-A FRAMEWORK

Authentication

- Multi-factor authentication (MFA)
- Secure password hashing (bcrypt)
- Credential validation controls

Authorization

- Role-based access control (RBAC)
- Least privilege enforcement
- Access policy restrictions

Accounting

- Centralized logging
- Audit trails
- Monitoring of access attempts

Green Pace

Unit Testing

Input Validation Controls

- Testing parameterized input handling
- Positive and negative test cases
- Google Test framework (C++)
- Automated vulnerability detection

Green Pace

INPUT VALIDATION IMPLEMENTATION

```
bool isValidUsername(const std::string& input) {
    if (input.empty()) return false;

    for (char c : input) {
        if (!isalnum(c)) {
            return false;
        }
    }
    return true;
}
```

UNIT TESTING WITH GOOGLE TEST

```
#include <gtest/gtest.h>

TEST(InputValidationTest, AcceptsValidInput) {
    EXPECT_TRUE(isValidUsername("Jessica123"));
}

TEST(InputValidationTest, RejectsSQLInjection) {
    EXPECT_FALSE(isValidUsername("' OR 1=1 --"));
}
```

Test 1
Does the function accept valid input?

```
TEST(InputValidationTest, AcceptsValidInput) {
    EXPECT_TRUE(isValidUsername("Jessica123"));
}
```

- Objective: Verify legitimate input is accepted
- Expected Result: TRUE
- Outcome: PASS

Test 2
Does the function reject SQL injection input?

```
TEST(InputValidationTest, RejectsSQLInjection) {
    EXPECT_FALSE(isValidUsername("' OR 1=1 --"));
}
```

- Objective: Detect injection attempt
- Expected Result: FALSE
- Outcome: PASS

Test 3
Does the function block script injection?

```
TEST(InputValidationTest, RejectsScriptInjection) {
    EXPECT_FALSE(isValidUsername("<script>alert('hack')</script>"));
}
```

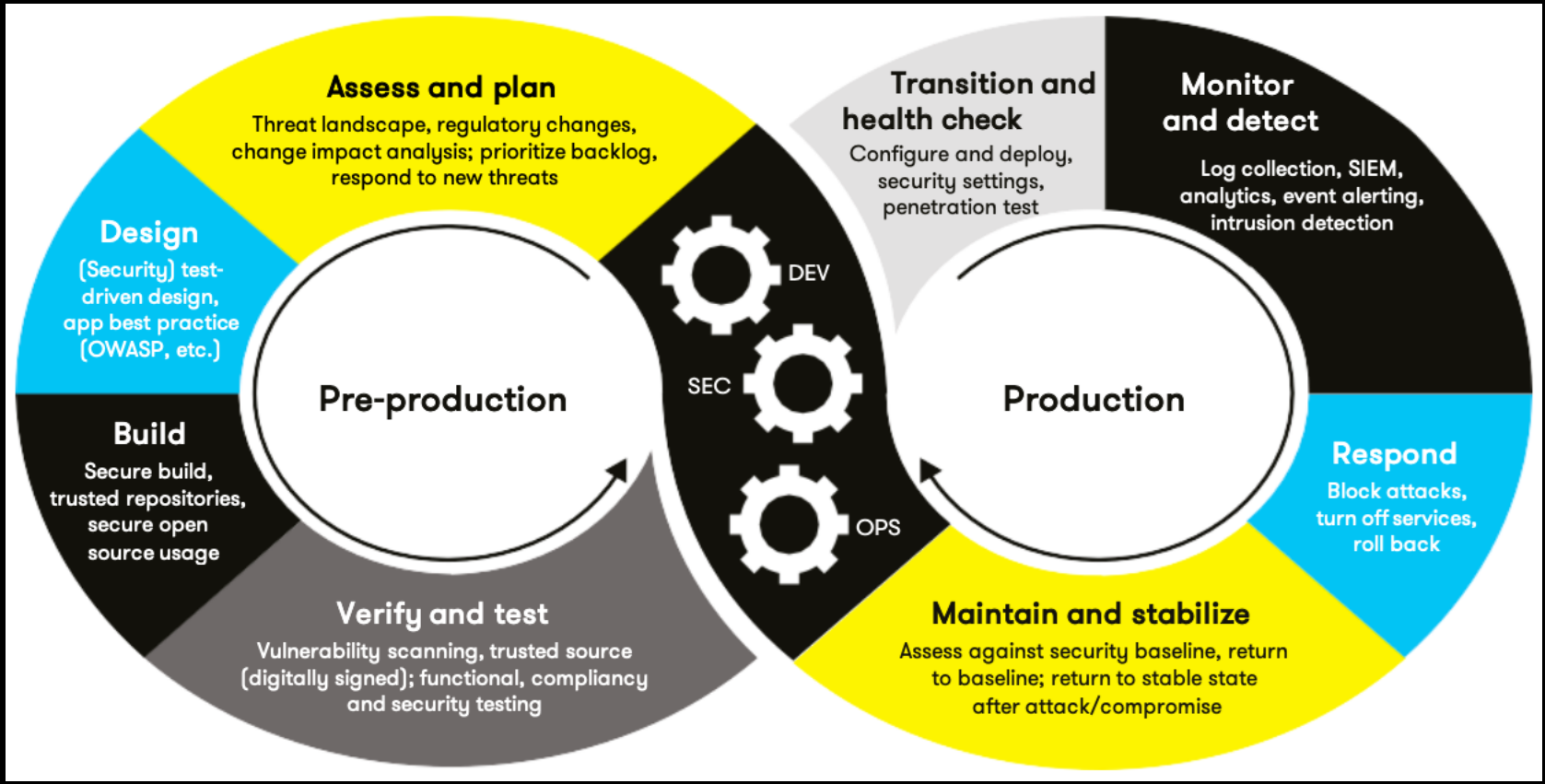
- Objective: Prevent cross-site scripting
- Expected Result: FALSE
- Outcome: PASS

Test 4
Does the function safely handle empty input?

```
TEST(InputValidationTest, RejectsEmptyInput) {
    EXPECT_FALSE(isValidUsername(""));
}
```

- Objective: Validate edge case handling
- Expeted Result: FALSE
- Outcome: PASS

AUTOMATION SUMMARY



Green Pace

TOOLS

- DevSecOps pipeline integrates security at every stage of development
- Static analysis and unit testing during build phase
- Code review before merge
- CI/CD security checks before deployment
- Monitoring and logging in production

Green Pace

RISKS AND BENEFITS

Problems Identified

- Injection vulnerabilities (SQL, XXS)
- Credential compromise risk
- Insufficient validation and monitoring

Risks and Limitations

- Increased development time
- Possible performance overhead
- Security controls require ongoing maintenance

Benefits of Acting Now

- Reduced breach likelihood
- Lower financial and reputational impact
- Stronger regulatory compliance

Recommended Next Steps

- Expand automated testing coverage
- Implement regular security audits
- Provide secure coding training

Green Pace

RECOMMENDATIONS

Identified Gaps

- Limited automated test coverage beyond input validation
- No formal secure coding training requirement
- Monitoring and logging policies not fully defined
- No documented incident response playbook

Recommended Enhancements

- Expand security testing across all input fields and APIs
- Implement recurring developer security training
- Define centralized logging and alert thresholds
- Establish formal incident response procedures

Green Pace

CONCLUSIONS

Standards to Adopt Moving Forward

- Secure coding standards aligned with OWASP guidelines
- Mandatory input validation and parameterized queries
- Multi-factor authentication and strong password hashing
- Encryption by default (TLS 1.2+, AES-256)
- Automated security testing in CI/CD pipelines
- Formal incident response and monitoring standards

Green Pace

REFERENCES

Google. (n.d.). *GoogleTest documentation*. https://google.github.io/googletest/

OWASP Foundation. (n.d.). *SQL injection prevention cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP Foundation. (n.d.). *Cross-site scripting (XSS) prevention cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

Green Pace